

Pre-Questionnaire Responses

- **In your 'AI-Powered Resume Screening' project, you used OCR and SpaCy for entity extraction. Can you describe a specific challenge you faced during the data cleaning phase and how you overcame it?**

Answer: Okay, One key challenge I faced was handling noisy and inconsistent OCR output, such as broken words, extra spaces, and unwanted characters from scanned resumes. This affected the accuracy of SpaCy's entity extraction. To solve this, I applied text preprocessing techniques like removing special characters, fixing spacing issues using regex, and normalizing the text. I also added custom rules and patterns in SpaCy to improve entity recognition. This significantly improved the overall extraction accuracy.

- **Which of the following Pandas methods is used to fill missing values with a specific constant or the result of a calculation?**

Answer: I will be used `df.fillna()` method to fill missing value with a specific constant.

`df.fillna()` is used to replace missing (NaN) values with a specific value or computed result.

- **You developed a RAG-based chatbot using Ollama and ChromaDB. How did you handle the trade-off between chunk size for context retention and the precision of vector search retrieval?**

Answer: In my RAG-based chatbot using Ollama and ChromaDB, managing chunk size was important because larger chunks improved context but reduced retrieval precision, while smaller chunks improved retrieval but lost context.

To handle this, I used chunk sizes of around 350 tokens with some overlap (80 tokens) to maintain continuity. I also used paragraph-based splitting instead of fixed sizes and tuned the chunking based on retrieval accuracy and response quality.

This helped improve document relevance and reduced hallucinations in responses.

- **Write a Python function that takes a list of integers and returns a new list containing only the even numbers, multiplied by 2?**

Answer: To solve this, I filter out only the even numbers from the list and then multiply each of them by 2 using a list comprehension.

```
def process_numbers(numbers):  
    return [num * 2 for num in numbers if num % 2 == 0]
```

This function checks each number, keeps only the even ones, and returns a new list with their values doubled.

- **In the context of Transformer models, what is the primary purpose of the 'Multi-Head Attention' mechanism?**

Answer: The primary purpose of Multi-Head Attention is to allow the model to focus on different parts of the input sequence simultaneously and capture multiple types of relationships. This helps the model better understand context by learning from different representation subspaces in parallel.

- **What is the output of the following code snippet? `def create_multipliers():
 return [lambda x: x * i for i in range(3)]
for multiplier in create_multipliers():
 print(multiplier(2))`?**

Answer:

```
def create_multipliers():  
    return [lambda x: x * i for i in range(3)]  
for multiplier in create_multipliers():  
    print(multiplier(2))
```

O/P:

4
4
4

This happens due to Python late binding behavior in closures. All the lambda functions reference the same variable `i`, and when they are executed, the loop has already completed with `i = 2`. Hence, each function returns $2 * 2 = 4$.